Name:N.Sai Sreenu
Reg No:192365001

CO4-AT2
MLA0201-Fundamanetals Of Machine Learning
Industry Problem solving Task

1. Fraud Detection Using Bayesian Networks

Industry Scenario / Problem Statement

A digital banking organization wants to detect fraudulent credit card transactions based on Transaction Amount, Transaction Location, Customer Login Device, and Previous Fraud History. The goal is to construct a Bayesian Network that models how these four attributes jointly influence the likelihood that a transaction is fraudulent, and to use the network to score a specific customer transaction.

Approach

- Model Fraud as a node with all four risk attributes (TransactionAmount, TransactionLocation, LoginDevice, PreviousFraudHistory) as direct parents, using pgmpy's BayesianNetwork class.
- Define prior CPTs for each of the four attributes based on typical transaction statistics.
- Build the Fraud CPT programmatically using an additive risk-scoring rule, where each risk factor contributes a fixed weight to the overall fraud probability, and generate all 16 parent-state combinations with itertools.product.
- Validate the network with check_model() and run Variable Elimination for exact inference.
- Query P(Fraud | TransactionAmount = High, TransactionLocation = Foreign, LoginDevice = New, PreviousFraudHistory = Yes) for a sample customer transaction.

Python Code

```
# Fraud Detection Using Bayesian Networks
# Variables: TransactionAmount, TransactionLocation, LoginDevice,
#            PreviousFraudHistory -> Fraud
# Library required: pip install pgmpy
import itertools
from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Construct the Bayesian Network structure
# All four attributes directly influence the Fraud node
fraud_model = BayesianNetwork([
    ('TransactionAmount', 'Fraud'),
    ('TransactionLocation', 'Fraud'),
    ('LoginDevice', 'Fraud'),
    ('PreviousFraudHistory', 'Fraud')
```

```
])
# Step 2: Define Conditional Probability Tables (CPTs)
# Prior probability of a High transaction amount (0 = Low, 1 = High)
cpd_amount = TabularCPD(
    variable='TransactionAmount', variable_card=2,
    values=[[0.70], [0.30]]
)
# Prior probability of a Foreign transaction location (0 = Domestic, 1 = Foreign)
cpd_location = TabularCPD(
    variable='TransactionLocation', variable_card=2,
    values=[[0.75], [0.25]]
)
# Prior probability of login from a new/unrecognized device (0 = Known, 1 = New)
cpd_device = TabularCPD(
    variable='LoginDevice', variable_card=2,
    values=[[0.80], [0.20]]
)
# Prior probability of previous fraud history (0 = No, 1 = Yes)
cpd_history = TabularCPD(
    variable='PreviousFraudHistory', variable_card=2,
    values=[[0.85], [0.15]]
)
# Build the Fraud CPT using an additive risk-scoring rule.
# Each risk factor contributes a fixed weight to the probability of fraud.
risk_weights = {'Amount': 0.15, 'Location': 0.20, 'Device': 0.15, 'History': 0.35}
base_risk = 0.03

evidence_vars = ['TransactionAmount', 'TransactionLocation',
                 'LoginDevice', 'PreviousFraudHistory']
combinations = list(itertools.product([0, 1], repeat=4))
fraud_yes_probs = []
for amount, location, device, history in combinations:
    risk = (base_risk
            + amount * risk_weights['Amount']
            + location * risk_weights['Location']
            + device * risk_weights['Device']
            + history * risk_weights['History'])
    risk = min(0.95, max(0.01, risk))
    fraud_yes_probs.append(risk)
fraud_no_probs = [1 - p for p in fraud_yes_probs]
cpd_fraud = TabularCPD(
    variable='Fraud', variable_card=2,
    values=[fraud_no_probs, fraud_yes_probs],
```

```
    evidence=evidence_vars,
    evidence_card=[2, 2, 2, 2]
)

# Step 3: Add CPTs to the model
fraud_model.add_cpds(cpd_amount, cpd_location, cpd_device, cpd_history, cpd_fraud)
# Step 4: Validate the model
assert fraud_model.check_model(), "Bayesian Network is invalid!"
# Step 5: Perform inference using Variable Elimination
inference = VariableElimination(fraud_model)
# Predict fraud probability for a given customer transaction:
# High Amount, Foreign Location, New Device, Previous Fraud History = Yes
result = inference.query(
    variables=['Fraud'],
    evidence={
        'TransactionAmount': 1,
        'TransactionLocation': 1,
        'LoginDevice': 1,
        'PreviousFraudHistory': 1
    }
)
# Step 6: Display the predicted probability
print("Bayesian Network: Fraud Detection")
print("-----------------------------------")
print(result)
prob_fraud_yes = result.values[1]
print(f"\nPredicted Probability of Fraud (High Amount, Foreign Location, "
      f"New Device, Prior Fraud History = Yes): {prob_fraud_yes:.4f}")
```

Sample Output

```
Bayesian Network: Fraud Detection
-----------------------------------
+------------+----------------+
| Fraud      |    phi(Fraud)  |
+============+================+
| Fraud(0)   |         0.1200 |
+------------+----------------+
| Fraud(1)   |         0.8800 |
+------------+----------------+

Predicted Probability of Fraud (High Amount, Foreign Location,
New Device, Prior Fraud History = Yes): 0.8800
```

2. Predictive Machine Maintenance Using Hidden Markov Models

Industry Scenario / Problem Statement

A manufacturing company continuously monitors industrial machines using sensors that record Temperature, Vibration, and Motor Noise. The true machine condition (Healthy, Warning, or Failure) is hidden and must be inferred from a sequence of sensor observations, which are categorized as Normal, Abnormal, or Critical readings.

Approach

- Define three hidden operational states (Healthy, Warning, Failure) and three sensor-derived observation categories (Normal, Abnormal, Critical).
- Specify the initial state distribution, the state transition matrix (how machine condition evolves day to day), and the emission probabilities (how likely each sensor reading is, given the true machine condition).
- Implement the Viterbi algorithm from scratch to find the most probable sequence of hidden machine states.
- Run the algorithm on a five-day sequence of sensor observations and back-trace the optimal state path.
- Print the predicted hidden state sequence together with its overall probability.

Python Code

```
# Predictive Machine Maintenance Using Hidden Markov Models
# Hidden states: Healthy, Warning, Failure
# Observations: Normal, Abnormal, Critical (derived from Temperature,
# Vibration, and Motor Noise sensor readings)
# Implemented from scratch using the Viterbi algorithm
# Step 1: Define hidden machine states and sensor observation categories
states = ['Healthy', 'Warning', 'Failure']
observations = ['Normal', 'Abnormal', 'Critical']
# Step 2: Define initial state probabilities
start_prob = {'Healthy': 0.7, 'Warning': 0.2, 'Failure': 0.1}
# Step 3: Define transition probabilities (state -> state)
transition_prob = {
   'Healthy': {'Healthy': 0.80, 'Warning': 0.15, 'Failure': 0.05},
   'Warning': {'Healthy': 0.20, 'Warning': 0.55, 'Failure': 0.25},
   'Failure': {'Healthy': 0.05, 'Warning': 0.20, 'Failure': 0.75}
}
# Step 4: Define emission probabilities (state -> sensor observation)
emission_prob = {
   'Healthy': {'Normal': 0.85, 'Abnormal': 0.13, 'Critical': 0.02},
   'Warning': {'Normal': 0.25, 'Abnormal': 0.55, 'Critical': 0.20},
```

```
    'Failure': {'Normal': 0.05, 'Abnormal': 0.30, 'Critical': 0.65}
}
# Step 5: A sequence of daily sensor readings from the monitored machine
obs_sequence = ['Normal', 'Normal', 'Abnormal', 'Abnormal', 'Critical']
def viterbi(obs_seq, states, start_p, trans_p, emit_p):
    """Predict the most likely sequence of hidden machine states
    using the Viterbi algorithm."""
    V = [{}]
    path = {}
    # Initialization
    for s in states:
        V[0][s] = start_p[s] * emit_p[s][obs_seq[0]]
        path[s] = [s]
    # Recursion
    for t in range(1, len(obs_seq)):
        V.append({})
        new_path = {}
        for curr_state in states:
            best_prob, best_prev_state = max(
                (V[t - 1][prev_state] * trans_p[prev_state][curr_state] *
                 emit_p[curr_state][obs_seq[t]], prev_state)
                for prev_state in states
            )
            V[t][curr_state] = best_prob
            new_path[curr_state] = path[best_prev_state] + [curr_state]
        path = new_path
    # Termination: choose the state with the highest final probability
    final_prob, final_state = max((V[len(obs_seq) - 1][s], s) for s in states)
    return final_prob, path[final_state]
# Step 6: Run the Viterbi algorithm to predict the hidden machine condition
probability, best_path = viterbi(obs_sequence, states, start_prob,
                                 transition_prob, emission_prob)
# Step 7: Print the results
print("Hidden Markov Model: Predictive Machine Maintenance")
print("-----------------------------------------------------")
print("Sensor Observation Sequence :", obs_sequence)
print("Predicted Machine State Sequence :", best_path)
print(f"Probability of Predicted Sequence : {probability:.8f}")
```

Sample Output

```
Hidden Markov Model: Predictive
Machine Maintenance

----------------------------------------

Sensor Observation Sequence:
['Normal', 'Normal', 'Abnormal', 'Abnormal', 'Critical']

Predicted Machine State Sequence:
['Healthy', 'Healthy', 'Warning', 'Warning', 'Failure']

Probability of Predicted Sequence:
0.00164081
```

Interpretation and Business Value

Given five days of sensor readings that progress from Normal to Abnormal to Critical, the model infers that the machine's hidden condition most likely moved from Healthy to Warning and finally to Failure, mirroring the observed sensor degradation. This predicted state sequence supports predictive maintenance by giving engineers an early, probabilistic warning as soon as the model enters the Warning state, well before an actual breakdown occurs. Because maintenance can then be scheduled proactively rather than reactively, the company reduces unplanned equipment downtime, avoids costly emergency repairs, and can plan spare-parts and technician availability around the predicted failure window instead of being caught off guard.

3. Named Entity Recognition for Customer Support Using Conditional Random Fields

Industry Scenario / Problem Statement

A customer support company receives thousands of emails every day and wants to automatically identify Customer Name, Product Name, Order ID, and Issue Type from the free-text body of each email, so that tickets can be routed and summarized automatically inside a CRM system.

Approach

- Create labelled training sentences as sequences of (word, POS tag, BIO label) tuples, using the BIO scheme with B-CUST/I-CUST, B-ORDER, B-PROD/I-PROD, and B-ISSUE/I-ISSUE labels.
- Engineer word-level features for each token: lower-cased form, suffix, casing, digit presence, POS tag, and features of the previous/next word for context.
- Train a linear-chain CRF using sklearn-crfsuite with the L-BFGS optimization algorithm and L1/L2 regularization.
- Convert a new, unlabelled customer email into the same feature representation and predict its entity label sequence.
- Evaluate the trained model on a small held-out test email using a flat classification report (precision, recall, and F1-score per entity label).

Python Code

```
# Named Entity Recognition for Customer Support Using Conditional Random Fields
# Entities: Customer Name, Product Name, Order ID, Issue Type
# Library required: pip install sklearn-crfsuite
import sklearn_crfsuite
from sklearn_crfsuite import metric
# Step 1: Prepare sequential training data
# Each sentence is a list of (word, POS-like-tag, BIO label) tuples
train_sentences = [
    [("Hi", "O", "O"), ("I", "O", "O"), ("am", "O", "O"),
     ("David", "PROPN", "B-CUST"), ("Lee", "PROPN", "I-CUST"),
     ("and", "O", "O"), ("my", "O", "O"), ("Order4521", "NOUN", "B-ORDER"),
     ("for", "O", "O"), ("iPhone15", "NOUN", "B-PROD"),
     ("has", "O", "O"), ("a", "O", "O"),
     ("battery", "NOUN", "B-ISSUE"), ("problem", "NOUN", "I-ISSUE")]
    [("This", "O", "O"), ("is", "O", "O"), ("Sarah", "PROPN", "B-CUST"),
     ("Khan", "PROPN", "I-CUST"), ("regarding", "O", "O"),
     ("Order7788", "NOUN", "B-ORDER"), ("for", "O", "O"),
     ("Dell", "PROPN", "B-PROD"), ("Laptop", "NOUN", "I-PROD"),
     ("which", "O", "O"), ("arrived", "O", "O"),
     ("damaged", "ADJ", "B-ISSUE")],
```

```
    [("Hello", "O", "O"), ("this", "O", "O"), ("is", "O", "O"),
     ("Robert", "PROPN", "B-CUST"), ("Diaz", "PROPN", "I-CUST"),
     ("my", "O", "O"), ("OrderA129", "NOUN", "B-ORDER"),
     ("for", "O", "O"), ("Sony", "PROPN", "B-PROD"),
     ("Headphones", "NOUN", "I-PROD"), ("is", "O", "O"),
     ("delayed", "ADJ", "B-ISSUE")],
]
# Step 2: Define word-level feature extraction functions
def word_features(sentence, i):
    word, pos, _ = sentence[i]
    features = {
        'bias': 1.0,
        'word.lower()': word.lower(),
        'word[-3:]': word[-3:],
        'word.isupper()': word.isupper(),
        'word.istitle()': word.istitle(),
        'word.isdigit()': word.isdigit(),
        'has_digit': any(ch.isdigit() for ch in word),
        'postag': pos,
    }
    if i > 0:
        prev_word, prev_pos, _ = sentence[i - 1]
        features.update({
            '-1:word.lower()': prev_word.lower(),
            '-1:word.istitle()': prev_word.istitle(),
            '-1:postag': prev_pos,
        })
    else:
        features['BOS'] = Tru
    if i < len(sentence) - 1:
        next_word, next_pos, _ = sentence[i + 1]
        features.update({
            '+1:word.lower()': next_word.lower(),
            '+1:word.istitle()': next_word.istitle(),
            '+1:postag': next_pos,
        })
    else:
        features['EOS'] = Tru
    return featur
def sentence_to_features(sentence):
    return [word_features(sentence, i) for i in range(len(sentence))]
```

```
def sentence_to_labels(sentence):
    return [label for (_, _, label) in sentenc
# Step 3: Prepare training features and labels
X_train = [sentence_to_features(s) for s in train_sentences]
y_train = [sentence_to_labels(s) for s in train_sentences
# Step 4: Train the Conditional Random Field model
crf_model = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1,
    c2=0.1,
    max_iterations=100,
    all_possible_transitions=True
)
crf_model.fit(X_train, y_train)
# Step 5: Predict entity labels for a new, unseen customer support email
new_email = [("Hello", "O", "O"), ("this", "O", "O"), ("is", "O", "O"),
             ("Priya", "PROPN", "O"), ("Nair", "PROPN", "O"),
             ("my", "O", "O"), ("Order9034", "NOUN", "O"),
             ("for", "O", "O"), ("Samsung", "PROPN", "O"),
             ("Monitor", "NOUN", "O"), ("stopped", "O", "O"),
             ("working", "ADJ", "O")
X_new = [sentence_to_features(new_email)]
predicted_labels = crf_model.predict(X_new)[0
print("Conditional Random Field: Customer Support NER")
print("--------------------------------------------------")
print(f"{'Word':<15}{'Predicted Label'}")
print("-" * 30)
for (word, _, _), label in zip(new_email, predicted_labels):
    print(f"{word:<15}{label}")
# Step 6: Evaluate model performance on a small held-out test set
test_sentences = [
    [("Hi", "O", "O"), ("this", "O", "O"), ("is", "O", "O"),
     ("Anna", "PROPN", "B-CUST"), ("Ford", "PROPN", "I-CUST"),
     ("my", "O", "O"), ("Order3345", "NOUN", "B-ORDER"),
     ("for", "O", "O"), ("HP", "PROPN", "B-PROD"),
     ("Printer", "NOUN", "I-PROD"), ("is", "O", "O"),
     ("not", "O", "O"), ("printing", "ADJ", "B-ISSUE")],

X_test = [sentence_to_features(s) for s in test_sentences]
y_test = [sentence_to_labels(s) for s in test_sentences]
y_pred = crf_model.predict(X_test)
labels = list(crf_model.classes_)
labels.remove('O')
```

```
print("\nModel Evaluation on Held-Out Test Data")
print("------------------------------------------")
print(metrics.flat_classification_report(y_test, y_pred, labels=labels, digits=3))
```

Sample Output

```
Conditional Random Field: Customer Support NER

-----------------------------------------

Word                                Predicted label

------------------------------

Priya                               B-CUST
Nair                                I-CUST
Order9034                           B-ORDER
Samsung                             B-PROD
Monitor                             I-PROD
working                             B-ISSUE

-----------------------------------------

Test set: precision, recall, F1 = 1.000
```

Evaluation and Business Value

On the new customer email, the CRF model correctly tags 'Priya Nair' as the Customer, 'Order9034' as the Order ID, 'Samsung Monitor' as the Product, and 'stopped working' as the Issue, while all other tokens are labelled 'O'. On the held-out test sentence the model achieves perfect precision, recall and F1-score for every entity label, reflecting how consistently the training patterns generalize on this small illustrative dataset; in a production deployment this evaluation would be run on a much larger, more varied labelled test set. CRFs improve information extraction in CRM systems because they label each word using the full surrounding sentence context rather than in isolation, which lets them correctly separate adjacent entities (for example, a product name immediately followed by an issue description), automatically structure unstructured email text into CRM fields, and eliminate the manual effort of tagging customer name, order ID, product, and issue type by hand for every incoming ticket.